

Examen Terminal 2 (Rattrapage) : Système distribué
Niveau 3 : GI - Année académique 2024/2025
Durée : 2H00 sans documents
Chargé de cours : Dr Abakar Mahamat Ahmat

Questions

1. Définition d'un système distribué
2. Expliquez comment une communication standard avec les sockets s'établit ?
3. Qu'est-ce que la sérialisation et pourquoi l'utilise-t-on ?
4. Quels types de représentations sont utilisés pour la sérialisation ?
5. Définition et principe du RPC
6. Définition et principe du RMI
7. Décrire le mécanisme d'échange reposant sur la technique de délégation appelée le design pattern proxy (interface, stub, skeleton, annuaire, ...)

Exercice 1 : Serveur Mono, ne traitant qu'un client

Écrire une classe `Serveur_Jouet` qui :

1. Crée un gestionnaire de socket sur le port 12000 (`ServerSocket`).
2. Attend une connexion entrante et associe à celle-ci un socket (`Socket`) quand elle arrive.
3. Lit de façon bloquante et ligne à ligne les données reçues et les affiche à l'écran avec un compteur de lignes (on utilisera les classes `InputStreamReader` `BufferedReader`).
4. On veillera à traiter au minimum les erreurs potentielles d'entrée sortie.

Afin de tester le fonctionnement on utilisera telnet (telnet 127.0.0.1 12000) pour se connecter au Serveur Jouet.

1. Que se passe-t-il si on lance plusieurs sessions telnet ?
2. Que deviennent les sessions telnet si on arrête le serveur ?
3. Que se passe-t-il si on lance une seconde instance du Serveur Jouet ?

Exercice 2 :

Nous disposons d'un service qui offre des opérations de gestion de son compte courant. Voici le code des méthodes offertes par ce service :

```
void debiter(double montant) {  
    }  
  
void crediter(double montant) {  
    }  
  
double lire_solde() {  
    }
```

1. On souhaite rendre chacune de ces méthodes accessibles à distance de manière à ce qu'elles définissent l'interface entre le client et le serveur. Écrire cette interface.
2. Dédurre la classe qui matérialise le service qui offre les opérations `debiter()`, `crediter()` et `lire_solde()`.
3. Compléter le fichier suivant : `Serveur.java` pour permettre l'enregistrement du service auprès de *RMI Registry*.


```

import java.rmi.*;
import java.rmi.server.*;

public class Serveur {
    public static void main(String[] args)
    {
        try {

            System.out.println("Serveur : Construction de l'implémentation");

            Compte cpt= new Compte(15.50);

            System.out.println("Objet Compte enregistré dans RMIregistry");

            // a completer

            System.out.println("Attente des invocations des clients ");
        }
        catch (Exception e) {
            System.out.println("Erreur de liaison de l'objet Compte");
            System.out.println(e.toString());
        }
    } // fin du main
} // fin de la classe

```

4. Quels sont tous les fichiers nécessaires à générer au lancement du Serveur.
5. Compléter le programme du client Client.java qui doit être lancé à partir d'un autre répertoire ou d'une autre machine.

```

import java.io.*;
import java.rmi.*;

class Client
{
    public static void main (String [] argv) throws IOException
    {
        if(argv.length != 2){

            System.out.println("Usage : java Client <nombre>
            <operation>");

            System.exit(1);

        }
        // operation = 1: credit, 2: dedit

        System.setSecurityManager(new RMISecurityManager());

        double valeur = Double.parseDouble(argv[0]);
        int operation = Integer.parseInt(argv[1]);
        try {

            // a completer : CompteInterface cpt = ...

            if (operation==1) cpt.crediter(valeur);

            if (operation ==2) cpt.debiter(valeur);

            System.out.println ("Votre solde courant = " + cpt.lire_solde() + " euros");

```

```
}catch (Exception e) {  
    System.out.println("Erreur d'accès à un objet distant");  
    System.out.println(e.toString());  
}  
}
```

Bonne chance